

**TOP 100  
AZURE  
DATA  
FACTORY  
INTERVIEW  
PREPARATION  
QUESTIONS AND  
ANSWERS  
3+ YOE**

**BASED ON REAL-TIME  
SCENARIOS, 2025**

## Section 1 – Core Azure Data Factory (ADF) Scenarios(1-30)

### Scenario 1 – Incremental Data Load Using Watermarking

**Description:**

Load only new or updated records from a SQL source to Azure Data Lake.

**Explanation:**

Use a high-watermark column such as `LastModifiedDate`. Store the latest value in a control table or file and use it in your next run.

**Example:**

```
SELECT * FROM Sales
WHERE LastModifiedDate > '@{pipeline().parameters.LastWatermark}'
```

**Flow:** Source → Copy Activity → ADLS → Update Watermark

**Key Benefit:** Loads delta data only, improving performance.

---

### Scenario 2 – Parameterizing Pipelines for Reusability

**Description:**

Make one pipeline handle multiple tables dynamically.

**Explanation:**

Add dataset parameters (`TableName`, `SinkPath`) and pass them through pipeline parameters using dynamic expressions.

**Example:** `@concat('/raw/', pipeline().parameters.TableName)`

**Flow:** Lookup → ForEach → Copy Activity

**Key Benefit:** Reduces duplication and simplifies maintenance.

---

### Scenario 3 – Failure Alerts via Logic App

**Description:**

Send an email if any activity fails.

**Explanation:**

Use an “On Failure” dependency with a Web Activity calling a Logic App endpoint.

**Flow:** ADF → Web Activity → Logic App → Email

**Key Benefit:** Immediate awareness of pipeline failures.

---

### Scenario 4 – Event and Schedule Triggers

**Description:**

Run daily loads at 2 AM or when new files arrive.

**Explanation:**

Configure both Schedule and Blob Event triggers pointing to the same pipeline.

**Key Benefit:** Hands-free automation and timely refreshes.

---

## Scenario 5 – Dynamic Table Loads Using Lookup and ForEach

**Description:**

Load multiple tables defined in a config file.

**Explanation:**

Read the config via Lookup, then ForEach loop over each record, executing Copy Activity with parameters.

**Flow:** Lookup → ForEach → Copy

**Key Benefit:** Scales easily for hundreds of tables.

---

## Scenario 6 – Error Handling with Custom Logging

**Description:**

Capture activity errors in a custom log table.

**Explanation:**

Use “On Failure” paths to execute a Stored Procedure Activity inserting pipeline name, activity name, and error message into SQL log table.

**Flow:** Activity → OnFailure → Stored Procedure

**Key Benefit:** Centralized error tracking for monitoring.

---

## Scenario 7 – Copying Data Between Different Regions

**Description:**

Move data from East US SQL DB to West Europe ADLS.

**Explanation:**

Use two self-hosted IRs and enable staging via Blob Storage to optimize cross-region transfer.

**Flow:** Source → Staging Blob → Sink

**Key Benefit:** Faster and secure inter-region transfers.

---

## Scenario 8 – Using Variables and Set Variable Activity

**Description:**

Maintain loop counters or dynamic file paths within a pipeline.

**Explanation:**

Initialize a variable and update its value inside loops using Set Variable Activity.

**Example:** @concat('/archive/', item().FileName)

**Key Benefit:** Better control flow and flexibility.

---

## Scenario 9 – Validating Input Files Before Processing

**Description:**

Ensure files follow correct naming and schema before copying.

**Explanation:**

Use Get Metadata Activity to check file size and name patterns, then If Condition Activity to proceed or fail.

**Flow:** Get Metadata → If Condition → Copy

**Key Benefit:** Prevents bad data loads.

---

## Scenario 10 – Data Flow for Complex Transformations

**Description:**

Perform joins, aggregations, and derived columns without Databricks.

**Explanation:**

Use ADF Mapping Data Flows with Source, Join, Aggregate, and Sink transformations.

**Flow:** ADLS → Data Flow → Synapse Table

**Key Benefit:** Low-code ETL within ADF for complex logic.

---

### Mini Interview Tips (for Scenarios 1–10)

- Stress the use of dynamic content and parameters.
  - Mention how you track incremental loads and errors.
  - Emphasize monitoring and automation via triggers and Logic Apps.
- 

## Scenario 11 – Data Archival After Successful Load

**Description:**

After ingesting data from the source, archive the original file to a backup folder.

**Explanation:**

Use a Copy Activity to move the file, followed by a Delete Activity to remove it from the incoming folder.

Configure “On Success” dependency from Copy → Delete.

**Flow:** ADLS /Input → Copy → ADLS /Archive

**Key Benefit:** Keeps source folders clean and prevents duplicate loads.

---

## Scenario 12 – Using Lookup for Dynamic SQL

**Description:**

Store complex SQL queries in a config table and execute them dynamically.

**Explanation:**

The Lookup Activity fetches SQL statements from control DB. Use

`@activity('Lookup1').output.firstRow.Query` inside Copy Activity.

**Flow:** Lookup (Config Table) → Copy Activity

**Key Benefit:** Externalizes logic for easier maintenance.

---

## Scenario 13 – Parallel Copy Execution

**Description:**

Run multiple Copy Activities in parallel for faster ingestion.

**Explanation:**

Use ForEach Activity with “Batch count” set > 1 to execute concurrent copies.

**Flow:** ForEach (Parallel) → Copy Activities

**Key Benefit:** Improves performance and reduces overall pipeline runtime.

---

## Scenario 14 – Implementing Retry Logic

**Description:**

Handle transient connectivity issues gracefully.

**Explanation:**

Set the “Retry” property in activities (e.g., 3 attempts, 30 seconds interval).

**Key Benefit:** Increases reliability for unstable network or service connections.

---

## Scenario 15 – Using Stored Procedure for Post-Load Validation

**Description:**

Validate target table row count after loading.

**Explanation:**

After Copy Activity, call a Stored Procedure Activity that checks `source count = target count`.

**Flow:** Copy → Stored Procedure (Validation)

**Key Benefit:** Ensures data integrity before marking job complete.

---

## Scenario 16 – Filter and Conditional Execution

**Description:**

Run only specific files that meet a condition.

**Explanation:**

Use “Filter Activity” on metadata to select items where, e.g., `FileSize > 0`. Pass result to ForEach.

**Flow:** Get Metadata → Filter → ForEach → Copy

**Key Benefit:** Avoids unnecessary processing of empty files.

---

## Scenario 17 – Dynamic Folder Creation in ADLS

**Description:**

Create a new folder structure each day automatically.

**Explanation:**

Use `@concat('/raw/', pipeline().parameters.LoadDate)` in Sink dataset.

**Key Benefit:** Organizes data by date for easier tracking and purging.

---

## Scenario 18 – Pipeline Failure Dependency Handling

**Description:**

Stop downstream pipelines if upstream fails.

**Explanation:**

Use “On Failure” dependencies or trigger pipelines with run-status checks through REST API.

**Key Benefit:** Prevents cascading errors and maintains data consistency.

---

## Scenario 19 – Custom Logging to Azure Monitor

**Description:**

Push pipeline run status and duration to Azure Log Analytics.

**Explanation:**

Call Azure Monitor Data Collector API using Web Activity with pipeline run details.

**Key Benefit:** Centralized monitoring and trend analysis for ADF runs.

---

## Scenario 20 – Data Validation Before Insert

**Description:**

Reject bad records before inserting into target.

**Explanation:**

Use Mapping Data Flow with Derived Column and Conditional Split transformations to separate valid and invalid data.

**Flow:** Source → Conditional Split → Sink (Valid/Invalid)

**Key Benefit:** Ensures clean data in target systems.

---

### Mini Interview Tips (Scenarios 11 – 20)

- Mention how you handle parallelism, error retries, and data validation.
  - Stress real-time logging and monitoring using Azure Monitor.
  - Explain how archival and folder automation help with data governance.
- 

## Scenario 21 – Incremental Data Load Using Watermark

**Description:**

Load only new or updated data instead of full load.

**Explanation:**

Maintain a control table with last load timestamp. Use it in the source query:

```
SELECT * FROM Sales WHERE ModifiedDate > @{{variables('LastWatermark')}}
```

Update watermark after load.

**Key Benefit:** Reduces data movement and improves efficiency.

---

## Scenario 22 – Parameterized Linked Services

**Description:**

Use a single Linked Service for multiple databases or environments.

**Explanation:**

Enable “Parameterize” in Linked Service and pass values like `ServerName`, `DBName` dynamically.

**Key Benefit:** Simplifies deployment and avoids duplicate configurations.

---

## Scenario 23 – Multiple File Merge Before Load

**Description:**

Combine multiple CSVs into one before ingestion.

**Explanation:**

Use Data Flow’s *Union* transformation or use Copy Activity with wildcard path (e.g., `input/*.csv`).

**Key Benefit:** Reduces number of small files and simplifies load management.

---

## Scenario 24 – File Arrival Trigger

**Description:**

Automatically trigger pipeline when a file lands in storage.

**Explanation:**

Use *Event-based trigger* with Azure Blob Storage event grid integration.

**Key Benefit:** Enables near real-time ingestion without manual intervention.

---

## Scenario 25 – Handling CSV with Variable Columns

**Description:**

Source CSVs may have different schema structures.

**Explanation:**

Enable “Skip line count = 1” and use *Schema Drift* in Data Flow. Add Derived Columns to standardize schema.

**Key Benefit:** Supports semi-structured ingestion flexibly.

---

## Scenario 26 – Securely Storing Credentials

**Description:**

Avoid hardcoding connection strings.

**Explanation:**

Integrate Azure Key Vault with Linked Services. Use `@Microsoft.KeyVault(SecretName)` reference.

**Key Benefit:** Enhances security and simplifies credential rotation.

---

## Scenario 27 – Dynamic Pipeline Execution from Master Pipeline

**Description:**

Execute child pipelines dynamically based on configuration.

**Explanation:**

Use Lookup to fetch pipeline names from config table, ForEach to iterate, and *Execute Pipeline Activity* to trigger.

**Key Benefit:** Enables metadata-driven orchestration.

---

## Scenario 28 – Sending Pipeline Alerts via Email

**Description:**

Notify team on failure or success.

**Explanation:**

Use Logic App or Web Activity to send email notification with run status.

**Key Benefit:** Improves visibility and proactive issue resolution.

---

## Scenario 29 – Data Transformation with Databricks

**Description:**

Perform heavy data cleansing and joins using Databricks Notebook.

**Explanation:**

Use *Databricks Notebook Activity* in ADF pipeline and pass parameters dynamically.

**Example:**

```
df = spark.read.csv(input_path)
df_clean = df.filter(df["amount"] > 0)
df_clean.write.parquet(output_path)
```

**Key Benefit:** Handles complex transformations at scale.

---

## Scenario 30 – Copying Data Between Subscriptions

**Description:**

Migrate data from one Azure subscription to another.

**Explanation:**

Use *Managed Identity* authentication and configure *Cross-Subscription Linked Services*.

**Flow:** ADF → Source Blob (Subscription A) → Target Blob (Subscription B)

**Key Benefit:** Simplifies enterprise-level data movement.

---

### Mini Interview Tips (Scenarios 21 – 30)

- Focus on **incremental load** and **metadata-driven pipelines**, as these are real-world must-haves.
  - Emphasize **Key Vault integration** for secure, enterprise-grade design.
  - Be ready to explain **Databricks collaboration** for advanced ETL.
-

## Section 2 – ADF + Databricks Integration Scenarios (31 to 50).

---

### Scenario 31 – ADF Triggering Databricks Notebook

**Description:**

ADF calls a Databricks notebook for transformation.

**Explanation:**

Use *Databricks Notebook Activity* in ADF with parameters for path, cluster, and notebook name.

**Example:**

ADF → Notebook → Reads data from ADLS → Writes cleaned parquet back.

**Key Benefit:** Combines orchestration (ADF) + transformation (Databricks) in one flow.

---

### Scenario 32 – Parameter Passing Between ADF and Databricks

**Explanation:**

Define parameters in Databricks notebook, e.g.:

```
dbutils.widgets.text("SourcePath", "")
src=dbutils.widgets.get("SourcePath")
```

ADF passes dynamic values through activity settings.

**Key Benefit:** Promotes reusable notebooks with environment-based inputs.

---

### Scenario 33 – Mounting ADLS in Databricks

**Explanation:**

Mount ADLS storage once for easy read/write:

```
dbutils.fs.mount(
  "wasbs://container@storage.blob.core.windows.net/",
  "/mnt/data")
```

**Benefit:** Simplifies data access for multiple notebooks.

---

### Scenario 34 – Incremental Load with Databricks Merge

**Explanation:**

Use Delta Lake's MERGE to update target incrementally:

```
MERGE INTO target USING source
ON target.id=source.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

**Benefit:** Fast upserts vs full refresh.

---

## Scenario 35 – ADF Pipeline to Run Databricks Job Cluster

**Explanation:**

ADF notebook activity can specify job cluster config (JSON).

Automatically creates and deletes cluster post-execution.

**Benefit:** Cost control and auto-scaling for ETL.

---

## Scenario 36 – Error Handling Between ADF and Databricks

**Explanation:**

Use try-except in Databricks and throw custom errors:

```
try:
    df=spark.read.csv(path)
except Exception as e:
    dbutils.notebook.exit(str(e))
```

ADF captures failure in *On Failure* path.

**Benefit:** Improves debugging and logging.

---

## Scenario 37 – Data Quality Check in Databricks

**Explanation:**

Notebook validates row counts and nulls before load.

**Example:**

```
bad=df.filter(df["amount"].isNull())
if bad.count()>0:
    raise Exception("Null values found")
```

**Benefit:** Prevents bad data from propagating.

---

## Scenario 38 – ADF Lookup to Fetch Dynamic Notebook Path

**Explanation:**

Store notebook paths in SQL config table.

Lookup activity + ForEach calls Databricks Notebook dynamically.

**Benefit:** Metadata-driven automation.

---

## Scenario 39 – Integrating Databricks Delta with Power BI

**Explanation:**

After ADF→Databricks pipeline creates Delta tables, connect Power BI directly to them using *Azure Synapse Serverless* or *ODBC*.

**Benefit:** Near real-time analytics on refreshed data.

---

## Scenario 40 – Partitioned Parquet Output

**Explanation:**

Databricks writes partitioned data:

```
df.write.partitionBy("year", "month").parquet(output)
```

ADF reads from dynamic folder path.

**Benefit:** Optimizes read performance and query pushdown.

---

## Scenario 41 – Automating Cluster Start/Stop via ADF

**Explanation:**

Use Databricks REST API in ADF Web activity to start/stop clusters before/after jobs.

**Benefit:** Cost optimization for ad-hoc processing.

---

## Scenario 42 – Notebooks Chaining Through ADF

**Explanation:**

ADF executes multiple Databricks notebooks sequentially using *Execute Notebook Activity*.

**Benefit:** Creates logical layered data processing flows (Bronze→Silver→Gold).

---

## Scenario 43 – ADF Calling Databricks REST API

**Explanation:**

Use Web activity to trigger Databricks jobs by API.

**Benefit:** Flexible for external triggers and CI/CD integration.

---

## Scenario 44 – Parameter File from Blob

**Explanation:**

Databricks reads JSON config from Blob:

```
config=json.load(open("/dbfs/mnt/config/pipeline.json"))
```

**Benefit:** Config-driven ETL design.

---

## Scenario 45 – Data Profiling Using Databricks Notebook

### Explanation:

Generate summary stats like count, distinct, nulls.

**Benefit:** Helps data validation before load to warehouse.

---

## Scenario 46 – Storing ADF Logs in Databricks

### Explanation:

ADF sends run details to ADLS → Databricks aggregates for monitoring dashboard.

**Benefit:** Centralized pipeline analytics.

---

## Scenario 47 – ADF and Databricks for Streaming Data

### Explanation:

ADF pulls batch files while Databricks handles Spark Structured Streaming for near real-time processing.

**Benefit:** Unified batch + stream architecture.

---

## Scenario 48 – Handling Schema Evolution in Delta Lake

### Explanation:

Enable `spark.databricks.delta.schema.autoMerge.enabled=true` for new columns.

**Benefit:** Automatic schema adaptation without pipeline failures.

---

## Scenario 49 – Databricks Job Failure Alert in ADF

### Explanation:

Capture `Activity Run Output` status → If failed, trigger Logic App email/SMS.

**Benefit:** Immediate notification for support teams.

---

## Scenario 50 – ADF + Databricks End-to-End ETL Example

### Flow:

Blob → ADF copy to ADLS → Databricks transform → ADF load to Synapse.

**Benefit:** Complete enterprise-grade modern data pipeline.

---

### ✅ Tips for Scenarios 31–50

- Stress how ADF and Databricks complement each other.
- Mention **Delta Lake**, **parameterization**, and **cost optimization** in answers.

- Be ready to draw flow diagrams if asked in interview.

---

## **Section 3 – ADF Real-Time & Error-Handling Scenarios (51–75)**

---

### **Scenario 51 – Real-Time Data Ingestion Using Event Trigger**

**Description:**

Automatically start pipelines when a new file lands.

**Explanation:**

Use *Event Grid trigger* on Blob container. It listens for file creation events and triggers ADF.

**Benefit:** Real-time, serverless ingestion—no need for manual scheduling.

---

### **Scenario 52 – Handling Late Arriving Files**

**Explanation:**

Use a Wait + Get Metadata activity loop that checks file existence.

If not found, waits 10 mins and retries.

**Benefit:** Avoids pipeline failure due to delayed files.

---

### **Scenario 53 – Retry Mechanism on Failure**

**Explanation:**

Configure activity-level *Retry count* and *Interval in seconds*.

**Example:** Retry Copy Activity 3 times if transient error occurs.

**Benefit:** Improves resiliency.

---

### **Scenario 54 – Error Logging to SQL Table**

**Explanation:**

On Failure path → Stored Procedure activity → Log pipeline name, activity, and error message.

**Benefit:** Centralized error repository for analysis.

---

### **Scenario 55 – Skipping Failed Files**

**Explanation:**

Loop through multiple files; use *If Condition* to skip file if Copy fails but continue pipeline.

**Benefit:** Avoids entire job failure due to one corrupt file.

---

## Scenario 56 – Timeout Handling

**Explanation:**

Set *Timeout* property for long-running activities like Copy/Databricks.

If exceeded, pipeline fails gracefully.

**Benefit:** Controls resource utilization.

---

## Scenario 57 – Validation Activity Before Load

**Explanation:**

Use Data Flow or Databricks to validate schema, nulls, duplicates before Copy.

**Benefit:** Prevents loading bad data into production systems.

---

## Scenario 58 – Capturing Row Counts

**Explanation:**

Store Source Count and Target Count in pipeline variables or SQL audit table.

Compare counts post-load for data completeness.

**Benefit:** Ensures integrity of transferred data.

---

## Scenario 59 – Parameter File for Dynamic Config

**Explanation:**

Read configuration JSON from Blob or SQL, then dynamically pass values to Linked Services, Datasets, and Pipelines.

**Benefit:** Single source of truth for environments.

---

## Scenario 60 – Email Notification for Errors

**Explanation:**

Use Web Activity to trigger Logic App → Send email with Run ID, Activity Name, and Error Message.

**Benefit:** Real-time alerting to support teams.

---

## Scenario 61 – Logging Success Runs

**Explanation:**

Add Stored Procedure activity at pipeline end → Log execution status as *Success*.

**Benefit:** End-to-end monitoring of all pipelines.

---

## Scenario 62 – Detecting Missing Files

**Explanation:**

Use Lookup activity with expected filenames from SQL config.  
Compare against *Get Metadata* results from Blob.

**Benefit:** Identifies missing feeds proactively.

---

## Scenario 63 – Dynamic Retry with Web Activity

**Explanation:**

Use a custom Azure Function triggered from ADF to handle variable retry logic based on error code.

**Benefit:** Smart error recovery.

---

## Scenario 64 – Capturing Pipeline Duration

**Explanation:**

Use system variables `@pipeline().RunStart` and `@utcNow()` to calculate duration.  
Log it in SQL table.

**Benefit:** Helps performance analysis.

---

## Scenario 65 – Archiving Processed Files

**Explanation:**

After successful load, use Copy Activity to move source file to `/archive/yyyy/MM/dd/`.  
Then delete original.

**Benefit:** Maintains clean folder structure.

---

## Scenario 66 – Dynamic Retry for Databricks Jobs

**Explanation:**

If Databricks notebook fails, catch error output → Retry after a delay → Stop after 3 attempts.

**Benefit:** Robust fault-tolerance for transient cluster issues.

---

## Scenario 67 – Real-Time CDC using Azure SQL and ADF

**Explanation:**

Enable Change Data Capture (CDC) on Azure SQL → Use ADF to fetch incremental data using timestamp or version columns.

**Benefit:** Near real-time incremental loads.

---

## Scenario 68 – Load Balancing Large File Transfers

**Explanation:**

Split large file into multiple chunks using *Copy parallelism* property.

**Benefit:** Faster throughput and efficient resource use.

---

**Scenario 69 – Using Stored Procedure for Business Validation****Explanation:**

ADF executes a stored procedure to check data rules before load.

If invalid → return error → pipeline stops.

**Benefit:** Integrates ADF with business validation logic.

---

**Scenario 70 – Retry at Activity Group Level****Explanation:**

Wrap multiple activities in an *Execute Pipeline Activity*.

Configure retry on this parent level.

**Benefit:** Simplifies error handling for grouped logic.

---

**Scenario 71 – Logging ADF Run Details in Blob****Explanation:**

Use Web Activity → Azure Function → Write JSON logs to Blob.

**Benefit:** Cloud-native logging without DB dependency.

---

**Scenario 72 – Handling Null Columns Dynamically****Explanation:**

Use *Derived Column* transformation in Data Flow to replace nulls with defaults.

Example: `iif(isNull(price),0,price)`

**Benefit:** Prevents downstream load issues.

---

**Scenario 73 – Debugging Copy Activity Performance****Explanation:**

Enable *Logging Level = Verbose* to analyze throughput and bottlenecks.

**Benefit:** Optimizes pipeline design for better speed.

---

**Scenario 74 – Using Global Parameters for Deployment**

**Explanation:**

Define environment-wide parameters (e.g., region, resource group) at factory level.

**Benefit:** Simplifies CI/CD and environment-specific deployments.

---

## Scenario 75 – Handling Duplicate Records in Load

**Explanation:**

Use Data Flow *Aggregate* or Databricks `dropDuplicates()` before loading to target.

**Benefit:** Ensures clean and unique data in destination.

---

 **Tips for Section 3:**

- Focus on **real-time triggers, retries, and logging mechanisms**.
  - Mention **row count audits** and **CDC for incremental loads**.
  - Interviewers love hearing about **self-healing pipelines** and **metadata-driven validation**.
- 

## **Section 4 – Advanced Enterprise & CI/CD Scenarios (76–100)**

---

### Scenario 76 – ADF CI/CD with Azure DevOps

**Description:**

Automate deployment between Dev, UAT, and Prod.

**Explanation:**

Use ADF's ARM Template export → Publish to DevOps repo → Create Release Pipeline to deploy via ARM Template.

**Benefit:** Promotes consistency and automation.

---

### Scenario 77 – Parameterizing Environment Variables

**Explanation:**

Use global parameters like `EnvironmentName` and conditional expressions:

```
@if(equals(pipeline().globalParameters.Environment, 'DEV'), 'DevStorage', 'ProdStorage')
```

**Benefit:** Single pipeline usable across all environments.

---

### Scenario 78 – Blue-Green Deployment for ADF

**Explanation:**

Maintain two factories (ADF-Blue, ADF-Green). Deploy new changes to Green; switch routing post-

validation.

**Benefit:** Zero-downtime deployments.

---

## Scenario 79 – ADF Integration with Git Repository

**Explanation:**

Connect ADF to GitHub/DevOps Git → Enable source control for pipelines and datasets.

**Benefit:** Versioning, rollback, and collaborative development.

---

## Scenario 80 – Metadata-Driven Pipeline Framework

**Explanation:**

Create master pipeline → Lookup config table (source, target, load type) → ForEach executes Copy/Transform dynamically.

**Benefit:** Scalable and reusable data framework.

---

## Scenario 81 – Dynamic Data Flow Creation

**Explanation:**

Use parameters in Data Flow for Source and Sink datasets.

**Benefit:** Single flow handles multiple data sources.

---

## Scenario 82 – ADF Integration with Synapse Dedicated Pool

**Explanation:**

ADF loads processed data into Synapse tables via *Copy Activity* with PolyBase option.

**Benefit:** Fast bulk loading of large volumes.

---

## Scenario 83 – ADF Integration with Azure Data Lake Gen2 Hierarchy

**Explanation:**

ADF dynamically writes data into hierarchical folders (*year/month/day*).

**Benefit:** Partitioned structure aids downstream analytics.

---

## Scenario 84 – Optimizing ADF Copy Performance

**Explanation:**

Use:

- *Staging* with Blob for SQL to Synapse

- *Parallel Copy* for large tables
  - *PolyBase* for Synapse loading
- Benefit:** Achieves up to 10x faster loads.
- 

## Scenario 85 – Custom Logging Using Azure Function

**Explanation:**

ADF calls an Azure Function to log pipeline metadata (start, end, status).

**Benefit:** Unified logging across ADF and external processes.

---

## Scenario 86 – Integrating ADF with Power Automate

**Explanation:**

Use Web Activity to call Power Automate flow for approval or manual validation step.

**Benefit:** Human-in-the-loop data validation.

---

## Scenario 87 – Orchestrating ML Model Execution

**Explanation:**

ADF triggers Azure Machine Learning Pipeline or Databricks notebook to run models post data prep.

**Benefit:** Automates ML lifecycle integration.

---

## Scenario 88 – Controlled Backfill Load

**Explanation:**

For historical data load, use pipeline parameters for start/end dates.

Loop through months and process each batch sequentially.

**Benefit:** Efficient backfill without overloading resources.

---

## Scenario 89 – ADF Integration with REST API

**Explanation:**

Use REST Linked Service + Copy Activity to extract data from APIs like Salesforce.

**Benefit:** Simplifies data ingestion from SaaS sources.

---

## Scenario 90 – Pagination in API Calls

**Explanation:**

Use dynamic URL:

```
@concat('https://api/data?page=', item())
```

Loop through pages using *Until Activity*.

**Benefit:** Handles large API data extraction.

---

## Scenario 91 – ADF Trigger Dependency

**Explanation:**

Configure *Trigger dependency* to run Pipeline-B only after successful completion of Pipeline-A.

**Benefit:** Manages sequence without manual scheduling.

---

## Scenario 92 – Using Managed Identity for Authentication

**Explanation:**

Enable Managed Identity → Assign Storage Blob/Data Factory Contributor role.

**Benefit:** Secure, password-less authentication.

---

## Scenario 93 – Parallel Execution of Pipelines

**Explanation:**

Run multiple data loads in parallel using *ForEach concurrency*.

**Benefit:** Reduces total runtime for multi-source loads.

---

## Scenario 94 – Conditional Execution of Pipelines

**Explanation:**

Use *If Condition Activity*:

```
@if(equals(variables('LoadType'), 'Full'), true, false)
```

Execute based on condition.

**Benefit:** Flexible logic control.

---

## Scenario 95 – Data Drift Handling in Data Flow

**Explanation:**

Enable *Allow Schema Drift* → ADF dynamically adapts to schema changes.

**Benefit:** No need to redesign pipelines for minor schema updates.

---

## Scenario 96 – Hierarchical JSON Flattening

**Explanation:**

Use Data Flow's *Flatten* transformation for nested JSONs.

**Example:** API returning nested orders and items.

**Benefit:** Simplifies complex JSON ingestion.

---

## Scenario 97 – ADF Integration with Logic Apps for SLA Monitoring

**Explanation:**

ADF triggers Logic App if pipeline exceeds expected duration.

**Benefit:** SLA alerting for delayed jobs.

---

## Scenario 98 – End-to-End Data Lineage Tracking

**Explanation:**

Log source, transformations, and destination details in metadata tables.

**Benefit:** Enhances transparency and auditability.

---

## Scenario 99 – Pipeline Run Analysis via REST API

**Explanation:**

Use ADF Management REST API to fetch pipeline run history for dashboarding.

**Benefit:** Monitoring and analytics integration.

---

## Scenario 100 – ADF Orchestration for Enterprise Data Lakehouse

**Flow:**

ADF orchestrates → Databricks processes data → Delta Lake stores results → Power BI visualizes → Alerts via Logic App.

**Benefit:** Full modern data stack automation.

---

**Tips for Section 4:**

- Emphasize **DevOps CI/CD, parameterization, metadata-driven design, and secure identity-based access.**
  - These topics are favorites in **senior-level interviews** for ADF & Azure Data Engineer roles.
-

Thank You...!!